

we can access the two bytes simultaneously (by using **key.i**) or the same two bytes individually (using **key.ch[0]** and **key.ch[1]**).

This is a frequent requirement while interacting with the hardware. i.e. sometimes we are required to access two bytes simultaneously and sometimes each byte individually. Faced with such a situation, using union is the answer, usually.

Perhaps you would be able to understand the union data type more thoroughly if we take a fresh look at the output of the above program. Here it is...

```
key.i = 512
key.ch[0] = 0
key.ch[1] = 2
```

Let us understand this output in detail. 512 is an integer, a 2 byte number. Its binary equivalent will be 0000 0010 0000 0000. We would expect that this binary number when stored in memory would look as shown below.

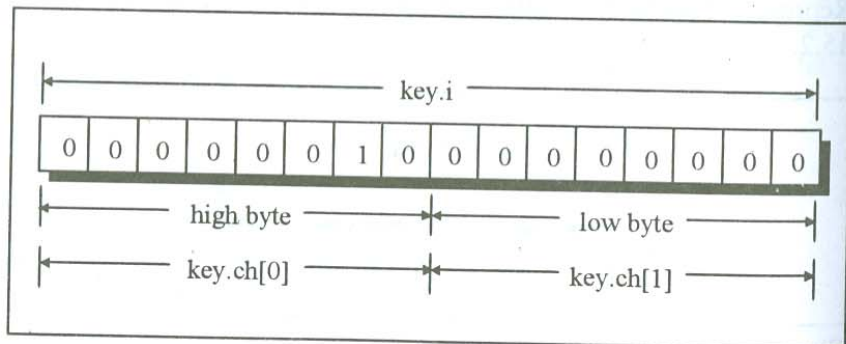


Figure 15.3

If the number is stored in this manner, then the output of **key.ch[0]** and **key.ch[1]** should have been 2 and 0. But, if you look at the output of the program written above, it is exactly the opposite. Why is it so? Because, in CPUs that follow little-endian

architecture (Intel CPUs, for example), when a two-byte number is stored in memory, the low byte is stored before the high byte. It means, actually 512 would be stored in memory as shown in Figure 15.4. In CPUs with big-endian architecture this reversal of bytes does not happen.

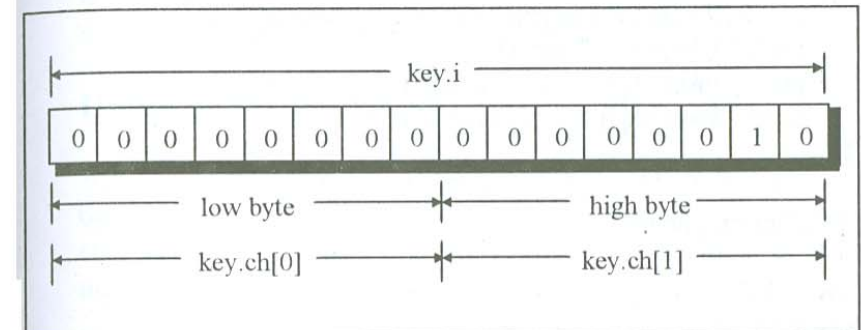


Figure 15.4

Now, we can see why value of **key.ch[0]** is printed as 0 and value of **key.ch[1]** is printed as 2.

One last thing. We can't assign different values to the different union elements at the same time. That is, if we assign a value to **key.i**, it gets automatically assigned to **key.ch[0]** and **key.ch[1]**. Vice versa, if we assign a value to **key.ch[0]** or **key.ch[1]**, it is bound to get assigned to **key.i**. Here is a program that illustrates this fact.

```
#include <stdio.h>
void main()
{
    union a
    {
        int i;
        char ch[2];
    };
    union a key;
```